dimension_stability: what happens when traces get longer

What this folder shows

Figure 1 of the paper uses CHES-CTF-2025 traces cut to 100, 200, 500, 1,000, 3,000 and 7,000 points. For each length it plots BI next to three other information measures (a mutual-information estimate, PI through a small network, and PI through a Gaussian model). BI stays flat and above zero. The other three drift to zero or below as the traces get longer, because they struggle with many input points.

The BI line comes from the Transformer models of guide 01, Level C. The other three lines come from this folder.

- `ches_nonbi_metric_cell.py` computes one value: one length, one random seed, one measure. It is built for clusters: first it writes a list of numbered tasks, then each task is run by its number and adds one row to a results file.
- `real_nonbi_dimcap_baselines.py` and `real_projected_bi_dimcap.py` are related checks from the long version.
- `multivariate_stability_driver.py` runs the same measures on artificial traces and needs no data.

Before you start

`CHES_Challenge.h5` in `datasets/raw/ches_ctf_2025/`, and TensorFlow for the network measure. One value takes seconds to minutes for the first two measures. The Gaussian measure is slow on long traces: about 8 minutes at 100 points, 40 minutes at 1,000, and 5 to 15 hours at 3,000 and 7,000. All 54 values behind Figure 1 take about 18 hours.

Run: two quick values

Write the task list for two measures at 100 points:

```
python -m dimension_stability.ches_nonbi_metric_cell --write-manifest-only \
  --dimensions 100 --seeds 0 --metrics GKOV_MI,MLP_PI \
  --manifest results/dimension_stability/manifest.csv --output-dir results/dimension_stability
cat results/dimension_stability/manifest.csv
```

```
$ python -m dimension_stability.ches_nonbi_metric_cell --write-manifest-only --dimensions 100 --seeds 0 --me
    trics GKOV_MI,MLP_PI --manifest results/dimension_stability/manifest.csv --output-dir results/dimension_
    stability && \
    cat results/dimension_stability/manifest.csv
Wrote 2 tasks to results/dimension_stability/manifest.csv
task_id,dataset,dimension,seed,metric,status,note
0,ches_ctf_2025,100,0,GKOV_MI,scheduled,CHES high-dimensional non-BI metric-split Plot B pass
1,ches_ctf_2025,100,0,MLP_PI,scheduled,CHES high-dimensional non-BI metric-split Plot B pass
[exit code 0, 2 s]
```

Run task 0 (the mutual-information estimate):

```
python -m dimension_stability.ches_nonbi_metric_cell --task-id 0 \
  --manifest results/dimension_stability/manifest.csv --output-dir results/dimension_stability
```

```
$ python -m dimension_stability.ches_nonbi_metric_cell --task-id 0 --manifest results/dimension_stability/ma
    nifest.csv --output-dir results/dimension_stability
```

```
[run] task=0 dataset=ches_ctf_2025 d=100 seed=0 metric=GKOV_MI
[done] dataset=ches_ctf_2025 d=100 seed=0 metric=GKOV_MI success=True value=-0.47667486389370806 elapsed=5.8
    71s
dimension_stability/ches_nonbi_metric_cell.py:197: FutureWarning: The behavior of DataFrame concatenation wi
    th empty or all-NA entries is deprecated. In a future version, this will no longer exclude e …
  df = pd.concat(
dimension_stability/ches_nonbi_metric_cell.py:162: FutureWarning: Downcasting object dtype arrays on .fillna
    , .ffill, .bfill is deprecated and will change in a future version. Call result.infer_object …
  success = group["success"].fillna(False).astype(bool) & values.notna()
[exit code 0, 7 s]
```

Run task 1 (PI through a small network) and show the results file:

```
python -m dimension_stability.ches_nonbi_metric_cell --task-id 1 \
  --manifest results/dimension_stability/manifest.csv --output-dir results/dimension_stability
cat results/dimension_stability/ches_nonbi_highdim_checkpoint.csv
```

```
$ python -m dimension_stability.ches_nonbi_metric_cell --task-id 1 --manifest results/dimension_stability/ma
    nifest.csv --output-dir results/dimension_stability && \
    cat results/dimension_stability/ches_nonbi_highdim_checkpoint.csv
[run] task=1 dataset=ches_ctf_2025 d=100 seed=0 metric=MLP_PI
[done] dataset=ches_ctf_2025 d=100 seed=0 metric=MLP_PI success=True value=-0.027576867668773986 elapsed=5.3
    71s
dataset,dimension,seed,metric,status,value,success,error,n_total_used,n_train,n_val,window_method,pi_method,
    source_artifact,started_at,finished_at,elapsed_sec,hostname,job_id,array_task_id
ches_ctf_2025,100,0,GKOV_MI,complete,-0.476674863893708,True,(MI subsampled 8000/24000),30000,24000,6000,cen
    ter,fast,"datasets/raw/ches_ctf_2025/CHES_Challenge.h5:Profiling_traces/traces[:,3450:3550]" …
ches_ctf_2025,100,0,MLP_PI,complete,-0.027576867668773986,True,,30000,24000,6000,center,fast,"datasets/raw/c
    hes_ctf_2025/CHES_Challenge.h5:Profiling_traces/traces[:,3450:3550]",2026-10-04T05:35:50+00: …
[exit code 0, 7 s]
```

For the whole figure, write the task list with `--dimensions 100,200,500,1000,3000,7000 --seeds 0,1,2 --metrics ePI_or_PI,MLP_PI,GKOV_MI` and run the 54 tasks, ideally several at a time.

Run: artificial traces

```
python -m dimension_stability.multivariate_stability_driver --dataset synthetic \
  --sweep-dimensions --dimensions 10,20,50 --n-seeds 2 --no-baselines \
  --output-dir results/dimension_stability/synthetic
```

```
$ python -m dimension_stability.multivariate_stability_driver --dataset synthetic --sweep-dimensions --dimen
    sions 10,20,50 --n-seeds 2 --no-baselines --output-dir results/dimension_stability/synthetic && \
    ls results/dimension_stability/synthetic
[... earlier output omitted; key lines kept ...]
[...]
======================================================================
MULTIVARIATE STABILITY SUMMARY
======================================================================
  synthetic                  BI: 1.0080 +/- 0.6921  (n=6)  M=3
    logistic: 0.0043
    lda: 0.0043
    mlp: 0.0040

Plots saved to results/dimension_stability/synthetic/figures
  synthetic: BI stability PASS (max CV = 0.1854)

Results saved to results/dimension_stability/synthetic
dimension_stability_results_20261004_053706.csv
dimension_stability_stats_20261004_053706.csv
figures

  d=10:   0%|          | 0/2 [00:00<?, ?it/s]
  d=10:  50%|█████     | 1/2 [00:06<00:06,  6.33s/it]
  d=10: 100%|██████████| 2/2 [00:12<00:00,  6.38s/it]
```

```
  d=20:   0%|          | 0/2 [00:00<?, ?it/s]
  d=20:  50%|█████     | 1/2 [00:04<00:04,  4.63s/it]
  d=20: 100%|██████████| 2/2 [00:10<00:00,  5.53s/it]


  d=50:   0%|          | 0/2 [00:00<?, ?it/s]
  d=50:  50%|█████     | 1/2 [00:22<00:22, 22.56s/it]
  d=50: 100%|██████████| 2/2 [00:45<00:00, 22.92s/it]

[exit code 0, 1 min 12 s]
```

What to look for

- The results file has one row per value: length, seed, measure, the value in bits, and the run time. A finished value is skipped on rerun; `--force` recomputes it.
- The mutual-information and Gaussian values come out the same on every machine, to three decimals. The network value moves by a few tenths of a bit between seeds and machines, because the network is trained for only a few seconds; treat one value on its own as noise around zero.
- All three measures stay at or below zero while BI stays between 0.3 and 0.6 bits. That contrast is Figure 1.